

LLM, Transformer, LoRA & Co.

Ein Lernskript für KI-Entwicklung am Beispiel eines lokalen ASM-Klassifikations-Use-Cases

Von den Grundlagen bis zu Training, LoRA, GGUF, Ollama, Evaluation und RAG.
Ohne unnötige Mathematik - mit vielen einfachen Bildern im Kopf.

Der rote Faden

Wir wollen Text automatisch in zwei Klassen einordnen: relevant oder nicht_relevant. Dafür testen wir ein Basismodell, trainieren mit LoRA nach, mergen das Ergebnis, konvertieren nach GGUF und laden es in Ollama. Dieses Dokument erklärt jeden Baustein dieses Weges.

Stand: Oktober 2026

Inhalt

1. Der Use Case in einem Satz
2. KI, Machine Learning, Deep Learning und Generative AI
3. Was ist ein LLM?
4. Token und Tokenizer
5. Embeddings
6. Transformer und Attention
7. Parameter, Gewichte und Modellgröße
8. Prompt, System Prompt und Kontextfenster
9. Inferenz und Sampling
10. Ollama, Hugging Face, Transformers und PyTorch
11. Fine-Tuning, PEFT und LoRA
12. LoRA-Parameter: r, alpha, dropout
13. Training: Epochs, Batch, Learning Rate, Loss
14. Adapter, Merge, Safetensors, GGUF und Quantisierung
15. Unser kompletter technischer Ablauf
16. Klassifikation und Evaluation
17. Overfitting, Underfitting und Datenqualität
18. RAG vs. Fine-Tuning vs. Prompting
19. CPU, GPU, RAM und VRAM
20. Typische Fehler und Debugging
21. Was in echten KI-Projekten zusätzlich dazugehört
22. Lernfragen mit Antworten
23. Kompakter Spickzettel

1. Der Use Case in einem Satz

Unser Lern-Use-Case ist eine vereinfachte Textklassifikation: Ein Text wird gelesen und das Modell soll genau eine von zwei Antworten liefern: "relevant" oder "nicht_relevant".

Bild im Kopf

Wie ein sehr schneller Sachbearbeiter: Du gibst ihm einen Text. Er liest ihn und legt ihn in Fach A oder Fach B. Das Sprachmodell ist der "Leser"; das Fine-Tuning bringt ihm bei, nach deinen Beispielen zu sortieren.

Was passiert technisch?

24. Ein Basismodell wie TinyLlama bekommt einen Text.
25. Das Modell erzeugt zunächst eine Antwort ohne spezielles Training.
26. Wir zeigen dem Modell Trainingsbeispiele mit korrekter Klasse.
27. Mit LoRA werden nur kleine Zusatzgewichte trainiert.
28. Der LoRA-Adapter wird mit dem Basismodell zusammengeführt.
29. Das komplette Modell wird in GGUF umgewandelt.
30. Ollama lädt das GGUF-Modell für lokale Inferenz.
31. Wir vergleichen Basismodell und trainierte Variante.

Text -> Tokenizer -> LLM -> Klasse

Training:

Beispiele + richtige Klasse -> LoRA -> trainierter Adapter

Deployment:

Basismodell + Adapter -> Merge -> GGUF -> Ollama

2. KI, Machine Learning, Deep Learning und Generative AI

Begriff	Einfach erklärt
Künstliche Intelligenz (KI)	Oberbegriff für Systeme, die Aufgaben lösen, für die Menschen normalerweise "Intelligenz" brauchen.
Machine Learning (ML)	Ein Teil von KI. Das System lernt Muster aus Daten statt nur festen Wenn-Dann-Regeln zu folgen.
Deep Learning	Ein Teil von ML mit großen neuronalen Netzen und vielen Schichten.
Generative AI	KI, die neue Inhalte erzeugt, z. B. Text, Bilder,

	Audio oder Code.
LLM	Large Language Model. Ein großes neuronales Netz, das Sprache verarbeitet und fortsetzt.

Merksatz

KI ist der große Schirm. ML ist ein Teil davon. Deep Learning ist ein Teil von ML. LLMs sind Deep-Learning-Modelle für Sprache.

Klassisches ML vs. LLM

Begriff	Einfach erklärt
Klassisches Text-ML	Text -> z. B. TF-IDF -> Zahlen -> Logistic Regression / SVM -> Klasse.
LLM	Text -> Token -> Transformer -> nächstes Token / Antwort. Kann zusätzlich per Prompt oder Fine-Tuning auf eine Aufgabe ausgerichtet werden.

3. Was ist ein LLM?

Ein LLM ist vereinfacht ein sehr großes Wahrscheinlichkeitsmodell für Sprache. Es bekommt eine Folge von Tokens und berechnet, welches Token als Nächstes wahrscheinlich ist. Durch sehr viele Trainingsdaten und sehr viele Parameter entstehen dabei Fähigkeiten wie Zusammenfassen, Klassifizieren, Übersetzen oder Programmieren.

Ganz einfach

Ein LLM "denkt" nicht wie ein Mensch. Es rechnet mit Zahlen und Wahrscheinlichkeiten. Seine Stärke entsteht daraus, dass es extrem viele Sprachmuster gelernt hat.

Causal Language Model

Unser Modell ist ein sogenanntes Causal Language Model. Es erzeugt Text von links nach rechts: Aus dem bisherigen Text wird jeweils das nächste Token vorhergesagt.

Eingabe: "Dieser Text ist"

Modell berechnet z. B.:

"relevant" 0.72

"nicht_relevant" 0.20

"interessant" 0.03

...

4. Token und Tokenizer

Ein Modell liest keine Wörter direkt. Ein Tokenizer zerlegt Text in kleine Einheiten - Tokens - und ordnet jedem Token eine Zahl zu.

```
"Ich hatte regelmäßig Kontakt."  
  |  
  v  
["Ich", " hatte", " regelmäßig", " Kontakt", "."]  
  |  
  v  
[1042, 581, 9421, 7344, 13]
```

Begriff	Einfach erklärt
Token	Kleine Textelemente. Ein Token kann ein ganzes Wort, ein Wortteil oder ein Zeichen sein.
Token-ID	Die Zahl, mit der das Modell intern ein Token bezeichnet.
Tokenizer	Software, die Text in Token-IDs und später Token-IDs wieder in Text umwandelt.
EOS Token	End-of-sequence. Markiert das Ende einer Ausgabe.
Padding	Füll-Tokens, damit mehrere Beispiele in einem Batch dieselbe Länge haben.
Attention Mask	Markiert, welche Token echte Daten und welche nur Padding sind.

Warum MAX_LENGTH wichtig ist

MAX_LENGTH = 128 bedeutet: Das Trainingsbeispiel darf höchstens 128 Tokens lang sein. Alles danach wird abgeschnitten. Ist der wichtige Teil hinten, kann das Training ihn dann gar nicht sehen.

5. Embeddings

Ein Embedding ist eine Zahlenliste, die Bedeutung repräsentiert. Wörter oder ganze Texte mit ähnlicher Bedeutung bekommen häufig Vektoren, die im mathematischen Raum näher beieinander liegen.

```
"Auto"    -> [ 0.12, -0.88, 0.41, ... ]  
"Fahrzeug" -> [ 0.15, -0.82, 0.45, ... ]  
"Banane"  -> [-0.51, 0.22, -0.73, ... ]
```

Bild im Kopf

Stell dir eine riesige Landkarte vor. Begriffe mit ähnlicher Bedeutung wohnen näher zusammen. Ein Embedding ist die Koordinate eines Textes auf dieser Landkarte.

Wo braucht man Embeddings?

- Ähnlichkeitssuche: Welcher gespeicherte Text passt zu meiner Frage?
- RAG: Relevante Dokumentabschnitte finden.
- Clustering: Ähnliche Texte automatisch gruppieren.
- Semantische Suche: nach Bedeutung statt nur nach exakt gleichen Wörtern.

6. Transformer und Attention

Der Transformer ist die grundlegende Netzwerkarchitektur moderner LLMs. Seine zentrale Idee ist Attention: Jedes Token kann berücksichtigen, welche anderen Tokens im Kontext gerade wichtig sind.

Beispiel

Satz: "Der Mann gab dem Jungen das Buch, weil er es brauchte." Das Modell muss verstehen, worauf "er" und "es" wahrscheinlich verweisen. Attention hilft dabei, Beziehungen zwischen Tokens zu gewichten.

Self-Attention

"Self" bedeutet: Die Tokens eines Textes schauen auf andere Tokens desselben Textes. Für jedes Token werden Beziehungen berechnet. Stark relevante Beziehungen bekommen mehr Gewicht.

```
Token "Kontakt"  
-> achtet stark auf "regelmäßig"  
-> achtet stark auf "bewaffnete Gruppe"
```

-> achtet wenig auf unwichtige Füllwörter

Q, K und V - warum q_proj und v_proj?

In Attention werden intern drei Repräsentationen verwendet: Query (Q), Key (K) und Value (V). Projektionen wie q_proj und v_proj sind trainierbare lineare Schichten, die diese Repräsentationen erzeugen. In unserem LoRA-Setup hängen wir kleine trainierbare LoRA-Matrizen an q_proj und v_proj.

Begriff	Einfach erklärt
Query (Q)	Vereinfacht: Wonach sucht dieses Token?
Key (K)	Vereinfacht: Welche Information biete ich an?
Value (V)	Vereinfacht: Welche Information wird übertragen?
q_proj / v_proj	Trainierbare Projektionen im Attention-Block. Gute Ansatzpunkte für LoRA.

Wichtig

Du musst die Matrixmathematik dahinter nicht auswendig können. Für die Praxis reicht zuerst: Transformer verarbeitet Kontext; Attention gewichtet Beziehungen; q_proj/v_proj sind Teile dieser Attention-Schichten.

7. Parameter, Gewichte und Modellgröße

Parameter sind die Zahlen, die ein neuronales Netz während des ursprünglichen Trainings gelernt hat. Man nennt sie oft Gewichte. Ein 1.1B-Modell besitzt ungefähr 1,1 Milliarden Parameter.

Begriff	Einfach erklärt
Parameter / Weight	Eine gelernte Zahl im neuronalen Netz.
1B	Eine Milliarde Parameter.
Base Model	Das ursprüngliche Modell vor deinem eigenen Fine-Tuning.
Checkpoint	Gespeicherter Zustand eines Modells zu einem bestimmten Trainingszeitpunkt.
Model Architecture	Der Bauplan: z. B. Llama, Qwen, Mistral. Nicht dasselbe wie die konkreten Gewichte.

Mehr Parameter = immer besser?

Nicht automatisch. Größere Modelle haben meist mehr Kapazität, brauchen aber mehr Speicher und Rechenleistung. Datenqualität, Training und passende Aufgabe sind genauso wichtig.

8. Prompt, System Prompt und Kontextfenster

Begriff	Einfach erklärt
Prompt	Die Eingabe, die du dem Modell gibst.
System Prompt	Übergeordnete Anweisung, z. B. Rolle, Regeln und erlaubte Ausgabe.
User Prompt	Die konkrete Benutzereingabe.
Context Window	Wie viele Tokens das Modell in einer Anfrage gleichzeitig berücksichtigen kann.
Chat Template	Modellspezifisches Format, das System-, User- und Assistant-Nachrichten in eine Tokenfolge umwandelt.

SYSTEM:

Du klassifizierst einen synthetischen ASM-Lerntext.
Antworte nur mit relevant oder nicht_relevant.

USER:

Ich hatte regelmäßig Kontakt zu einer bewaffneten Gruppe.

Prompt Engineering

Prompt Engineering bedeutet, Eingaben so zu formulieren, dass das Modell möglichst zuverlässig die gewünschte Aufgabe versteht. Gute Prompts sind eindeutig, enthalten klare Regeln und - falls nötig - Beispiele.

Für Klassifikation

Je weniger Freiraum du dem Modell gibst, desto besser. "Antworte ausschließlich mit genau einer dieser beiden Klassen" ist besser als "Bewerte den Text".

9. Inferenz und Sampling

Inferenz bedeutet: Das bereits trainierte Modell wird benutzt, um eine Ausgabe zu erzeugen. Dabei werden keine Gewichte verändert.

Begriff	Einfach erklärt
Inference / Inferenz	Modell anwenden, nicht trainieren.
Temperature	Steuert Zufälligkeit. Niedrig = deterministischer, hoch = kreativer.
Top-p	Beschränkt die Auswahl auf die wahrscheinlichsten Tokens, die zusammen eine bestimmte Wahrscheinlichkeitsmasse abdecken.
Top-k	Beschränkt die Auswahl auf die k wahrscheinlichsten nächsten Tokens.
num_predict / max_new_tokens	Maximale Anzahl neu erzeugter Tokens.
Seed	Kann Zufallsentscheidungen reproduzierbarer machen, wenn die Laufzeit es unterstützt.

Für deinen Klassifikator

Temperature = 0 und sehr wenige Output-Tokens sind sinnvoll. Du willst keine Kreativität, sondern möglichst stabil genau ein Label.

10. Ollama, Hugging Face, Transformers und PyTorch

Begriff	Einfach erklärt
Ollama	Lokale Laufzeit zum Laden und Ausführen von Sprachmodellen. In unserem Projekt vor allem für Inferenz.
Hugging Face	Ökosystem für Modelle, Datensätze und ML-Bibliotheken.
Transformers	Python-Bibliothek zum Laden und Verwenden vieler Transformer-Modelle.
PEFT	Hugging-Face-Bibliothek für Parameter-Efficient Fine-Tuning, z. B. LoRA.
PyTorch	Deep-Learning-Framework. Führt Tensorberechnungen aus und berechnet Gradienten für Training.
Safetensors	Dateiformat für Modellgewichte. Schnell und

	sicherer als beliebige Python-Pickle-Dateien.
GGUF	Dateiformat für effiziente lokale Inferenz, besonders im llama.cpp-/Ollama-Umfeld.
llama.cpp	C/C++-Projekt für effiziente lokale Inferenz und Werkzeuge wie die GGUF-Konvertierung.

In unserem Projekt

PEFT + Transformers + PyTorch trainieren lokal. Ollama führt das fertige Modell aus. Das sind zwei verschiedene Aufgaben.

11. Fine-Tuning, PEFT und LoRA

Fine-Tuning

Fine-Tuning bedeutet: Ein bereits vortrainiertes Modell wird mit eigenen Beispielen weitertrainiert, damit es sich besser an eine bestimmte Aufgabe, Sprache, Domäne oder Antwortform anpasst.

Bild im Kopf

Das Basismodell hat schon "Schule und Studium" hinter sich. Fine-Tuning ist eine zusätzliche Fachschulung für deine konkrete Aufgabe.

Warum nicht alle Milliarden Parameter trainieren?

Full Fine-Tuning verändert sehr viele Modellparameter. Das braucht viel Speicher, Rechenleistung und Trainingszeit. PEFT versucht deshalb, nur einen kleinen zusätzlichen Teil zu trainieren.

PEFT

PEFT steht für Parameter-Efficient Fine-Tuning. Es ist der Oberbegriff für Methoden, bei denen nur ein kleiner Anteil zusätzlicher oder ausgewählter Parameter trainiert wird.

LoRA

LoRA steht für Low-Rank Adaptation. Statt eine große Gewichtsmatrix vollständig zu verändern, werden kleine Zusatzmatrizen gelernt. Diese beeinflussen das Basismodell, benötigen aber viel weniger trainierbare Parameter.

Full Fine-Tuning:

riesige Modellmatrix W -> sehr viele Werte ändern

LoRA:

W bleibt weitgehend unverändert
+ kleine Matrix A
+ kleine Matrix B
= gezielte Anpassung mit viel weniger trainierbaren Werten

Adapter

Die gespeicherten LoRA-Zusatzgewichte nennt man oft Adapter. Er ist klein im Vergleich zum vollständigen Basismodell.

12. LoRA-Parameter: r, alpha, dropout

Begriff	Einfach erklärt
LORA_R / rank	Kapazität der kleinen LoRA-Matrizen. Höher = mehr trainierbare Information, aber auch mehr Speicher und Overfitting-Risiko.
LORA_ALPHA	Skaliert den Einfluss der LoRA-Anpassung. Häufig wird alpha ungefähr passend zum rank gewählt, z. B. r=16, alpha=32.
LORA_DROPOUT	Während des Trainings wird ein kleiner Teil der LoRA-Aktivierung zufällig ausgelassen. Das kann Overfitting reduzieren.
target_modules	Welche Modellschichten LoRA erhalten, z. B. q_proj und v_proj.

Beispiel

r=8 ist klein und sparsam. r=16 gibt LoRA mehr Lernkapazität. Das bedeutet aber nicht automatisch bessere Ergebnisse. Entscheidend ist die Evaluation auf unabhängigen Beispielen.

13. Training: Epochs, Batch, Learning Rate, Loss

Begriff	Einfach erklärt
Epoch	Ein kompletter Durchlauf durch alle Trainingsbeispiele.
Batch Size	Wie viele Beispiele gleichzeitig verarbeitet werden.

Gradient	Richtung, in die Gewichte verändert werden sollten, um den Fehler zu reduzieren.
Gradient Accumulation	Mehrere kleine Batches sammeln Gradienten, bevor ein Update erfolgt. Spart Speicher.
Learning Rate	Schrittgröße der Gewichtsänderung. Zu groß = instabil; zu klein = sehr langsam.
Loss	Zahl für den Trainingsfehler. Das Training versucht, den Loss zu senken.
Optimizer	Algorithmus, der mit den Gradienten die trainierbaren Parameter aktualisiert.
Training Step	Ein Optimizer-Schritt bzw. ein Teil des Trainingsablaufs.

100 Beispiele
 BATCH_SIZE = 1
 GRADIENT_ACCUMULATION = 4

=> ungefähr 4 Beispiele tragen gemeinsam zu einem Gewichtsupdate bei.

Was macht backpropagation?

Das Modell erzeugt eine Vorhersage. Dann wird berechnet, wie falsch sie ist. Über Backpropagation wird der Fehler rückwärts durch das Netz verfolgt. Daraus entstehen Gradienten. Der Optimizer nutzt sie für kleine Gewichtsänderungen.

Für Dumme in einem Satz

Vorhersagen -> Fehler messen -> herausfinden, welche Stellschrauben schuld waren -> Stellschrauben ein kleines Stück drehen -> wiederholen.

14. Adapter, Merge, Safetensors, GGUF und Quantisierung

Begriff	Einfach erklärt
LoRA Adapter	Nur die kleinen zusätzlich trainierten LoRA-Gewichte.
Merge	Basismodell und LoRA-Anpassung werden zu einem vollständigen Modell zusammengeführt.
Safetensors	Format, in dem Transformers-Modellgewichte häufig gespeichert werden.

GGUF	Format, das lokale Engines wie llama.cpp und Ollama sehr gut verarbeiten können.
Quantisierung	Gewichte werden mit weniger Bits gespeichert, z. B. 16 Bit -> 8/4 Bit. Spart Speicher und beschleunigt oft lokale Inferenz, kann aber etwas Qualität kosten.
FP32 / FP16	32- bzw. 16-Bit-Fließkommazahlen. FP16 braucht ungefähr halb so viel Speicher wie FP32 für die Gewichte.
Q4 / Q5 / Q8	Typische quantisierte GGUF-Varianten mit grob 4, 5 oder 8 Bit pro Gewicht, je nach Verfahren.

```

Basismodell
+
LoRA Adapter
|
v
MERGE
|
v
merged_model (Safetensors)
|
v
convert_hf_to_gguf.py
|
v
trained-model.gguf
|
v
ollama create
|
v
trained-modell:latest

```

Warum brauchten wir GGUF?

Der direkte Safetensors-Import verursachte in deinem Windows-Ollama-Setup Probleme. Über GGUF konnten wir das gemergte Modell sauber in Ollama registrieren.

15. Unser kompletter technischer Ablauf

A. Vor dem Training

- Das Basismodell liegt lokal als Transformers-/Safetensors-Modell vor.
- Die Trainingsdaten liegen als CSV mit text,label vor.
- Der Tokenizer gehört zum selben Modell wie die Trainingsgewichte.
- Ollama enthält eine passende Basismodell-Variante für Vorher-Nachher-Tests.

B. Beim Training

32. CSV laden und Labels validieren.
33. Chat Prompt aus System Prompt und Text bauen.
34. Prompt und Zielantwort tokenisieren.
35. Prompt-Tokens im Loss mit -100 maskieren, damit hauptsächlich die gewünschte Antwort gelernt wird.
36. LoRA an ausgewählte Attention-Schichten hängen.
37. Trainer startet mehrere Epochen.
38. LoRA-Adapter speichern.

C. Nach dem Training

39. LoRA mit dem Basismodell mergen.
40. Gemergtes Modell als Safetensors speichern.
41. Automatisch nach GGUF konvertieren.
42. Automatisch ein Ollama Modelfile erzeugen.
43. Automatisch ollama create ausführen.
44. Trainiertes Modell lokal testen und mit Basismodell vergleichen.

Wichtig für die Architektur

Training und Inferenz sind getrennte Phasen. PEFT/PyTorch trainiert. Ollama serviert anschließend das fertige Modell für Anfragen.

16. Klassifikation und Evaluation

Ein Modell ist nicht gut, nur weil einzelne Testfragen gut aussehen. Du brauchst systematische Evaluation auf Beispielen, die das Training nicht gesehen hat.

Begriff	Einfach erklärt
Ground Truth	Die fachlich korrekte Zielklasse für ein Beispiel.
Train Set	Daten, mit denen das Modell lernt.
Validation Set	Daten zur Auswahl von Einstellungen während der Entwicklung.

Test Set	Unberührte Daten für die finale Bewertung.
True Positive	Relevant ist wirklich relevant und wird relevant erkannt.
False Positive	Nicht relevant wird fälschlich als relevant markiert.
False Negative	Relevant wird übersehen und als nicht relevant markiert.
True Negative	Nicht relevant wird korrekt als nicht relevant erkannt.

Begriff	Einfach erklärt
Accuracy	Anteil aller richtigen Vorhersagen.
Precision	Von allem, was als relevant markiert wurde: Wie viel war wirklich relevant?
Recall	Von allem wirklich Relevanten: Wie viel wurde gefunden?
F1 Score	Kompromiss zwischen Precision und Recall.
Confusion Matrix	Tabelle mit TP, FP, FN und TN. Sehr nützlich für Fehleranalyse.

In sicherheitsrelevanten Use Cases

Welche Fehlerart teurer ist, ist eine fachliche Entscheidung. Hoher Recall reduziert übersehene relevante Fälle, kann aber mehr False Positives erzeugen. Deshalb darf die Metrik nicht nur technisch gewählt werden.

LLM-Ausgabe normalisieren

Bei generativer Klassifikation solltest du die Ausgabe technisch prüfen. Erlaubt sind nur die zwei Labels. Antworten wie "Der Text ist relevant, weil ..." sollten entweder verhindert oder robust normalisiert werden.

17. Overfitting, Underfitting und Datenqualität

Begriff	Einfach erklärt
Overfitting	Das Modell merkt sich Trainingsbeispiele zu stark und ist auf neuen Texten schlecht.
Underfitting	Das Modell lernt die Aufgabe nicht ausreichend.
Generalization	Fähigkeit, auf neue, unbekannte Beispiele

	sinnvoll zu reagieren.
Data Leakage	Information aus Test-/Zieldaten gelangt versehentlich ins Training. Ergebnisse wirken dann besser als sie wirklich sind.
Class Imbalance	Eine Klasse kommt viel häufiger vor als die andere.
Label Noise	Trainingslabels sind falsch, uneinheitlich oder widersprüchlich.

Warum 100 Beispiele schnell zu wenig sind

Bei 100 Beispielen kann ein Modell Formulierungen auswendig lernen, statt ein stabiles fachliches Muster zu lernen. Vielfalt ist oft wichtiger als einfach nur dieselben Satzmuster zu vervielfachen.

Gute Trainingsdaten

- fachlich korrekt gelabelt
- viele unterschiedliche Formulierungen
- Grenzfälle und schwierige Gegenbeispiele
- keine Duplikate zwischen Train und Test
- realistische Klassenverteilung oder bewusst gewählte Balance
- klare Labeldefinitionen, die Menschen reproduzierbar anwenden können

18. RAG vs. Fine-Tuning vs. Prompting

Begriff	Einfach erklärt
Prompting	Du änderst nur die Anweisung. Das Modell selbst bleibt unverändert.
Few-Shot Prompting	Du gibst einige Beispiele direkt im Prompt mit.
RAG	Vor der Antwort werden passende externe Dokumente gesucht und als Kontext an das LLM gegeben.
Fine-Tuning	Modellgewichte oder Adapter werden trainiert. Verhalten wird dauerhaft angepasst.
LoRA Fine-Tuning	Ressourcensparende Fine-Tuning-Variante mit kleinen Adaptern.

Wann was?

Begriff	Einfach erklärt
Wissen ändert sich häufig	RAG ist oft besser, weil Dokumente aktualisiert werden können, ohne das Modell neu zu trainieren.
Ausgabeformat / Stil / Verhalten lernen	Fine-Tuning kann sinnvoll sein.
Kleine Aufgabe und klare Regeln	Erst Prompting ausprobieren.
Eigene Fachinformationen zitierbar beantworten	RAG ist häufig passender.
Klassifikation mit festen Labels	Klassisches ML, Prompting oder Fine-Tuning vergleichen - die einfachste gute Lösung gewinnt.

Wichtiger Engineering-Grundsatz

Nicht jede KI-Aufgabe braucht ein LLM. Ein kleiner klassischer Klassifikator kann billiger, schneller und leichter erklärbar sein.

19. CPU, GPU, RAM und VRAM

Begriff	Einfach erklärt
CPU	Allgemeiner Prozessor. Kann LLMs ausführen und kleine Modelle trainieren, aber meist langsamer als eine geeignete GPU.
GPU	Prozessor mit sehr vielen parallelen Recheneinheiten. Besonders gut für Matrixoperationen im Deep Learning.
RAM	Arbeitsspeicher des Computers.
VRAM	Speicher direkt auf der GPU.
CUDA	NVIDIA-Plattform, über die Frameworks wie PyTorch NVIDIA-GPUs nutzen können.
Device	Ort der Berechnung, z. B. CPU oder CUDA-GPU.

Warum Modellgröße zählt

Ein 1.1B-Modell ist für lokale Experimente deutlich leichter als ein 7B- oder 70B-Modell. Training braucht zusätzlich Speicher für Aktivierungen, Gradienten und Optimizer-Zustände.

20. Typische Fehler und Debugging

Begriff	Einfach erklärt
404 von Ollama	Ollama läuft, aber das angefragte Modell ist meist nicht unter diesem Namen registriert.
Out of memory	Modell/Batch/Context ist zu groß für verfügbaren RAM oder VRAM.
Tokenizer mismatch	Tokenizer und Modellversion passen nicht zusammen. Ergebnisse oder Laden können scheitern.
Unsupported architecture	Importtool unterstützt die Modellarchitektur nicht.
Dependency conflict	Python-Pakete erwarten inkompatible Versionen, z. B. TensorFlow/protobuf.
Schlechte Modellqualität	Kann von Modellgröße, Daten, Labels, Prompt, Hyperparametern oder Evaluation kommen - nicht automatisch von einem "Bug".

Debugging-Reihenfolge

45. Fehlertext genau lesen: Training, Konvertierung, Import oder Inferenz?
46. Prüfen, welche Datei und welches Modell tatsächlich verwendet werden.
47. Kleinsten isolierten Test machen: z. B. ollama list oder ein einzelner Python-Import.
48. Erst danach größere Teile neu trainieren oder neu installieren.
49. Änderungen einzeln vornehmen, damit klar bleibt, was geholfen hat.

21. Was in echten KI-Projekten zusätzlich dazugehört

Begriff	Einfach erklärt
MLOps	Praktiken und Werkzeuge, um ML-Modelle reproduzierbar zu trainieren, zu versionieren, bereitzustellen und zu überwachen.
Model Registry	Verzeichnis mit freigegebenen Modellversionen und Metadaten.
Experiment Tracking	Speichert Parameter, Datenversion, Codeversion und Ergebnisse von Trainingsläufen.
Data Pipeline	Automatisierter Weg von Rohdaten zu bereinigten Trainings-/Inferenzdaten.
Monitoring	Überwacht Laufzeit, Fehler, Latenz und fachliche Modellqualität.

Data Drift	Eingabedaten verändern sich gegenüber dem Training.
Concept Drift	Die fachliche Beziehung zwischen Eingabe und Ziel verändert sich.
Retraining	Modell wird mit neuen validierten Daten erneut trainiert.
Human in the Loop	Menschen prüfen oder korrigieren Modellvorschläge.
Guardrails	Technische/fachliche Regeln, die Ein- oder Ausgaben begrenzen und validieren.
Latency	Zeit von Anfrage bis Antwort.
Throughput	Wie viele Anfragen pro Zeiteinheit verarbeitet werden können.
API	Programmierschnittstelle, über die andere Systeme das Modell aufrufen.
Endpoint	Konkrete Netzwerkadresse einer API-Funktion.
Deployment	Ein Modell so bereitstellen, dass andere Anwendungen es nutzen können.
Versionierung	Nachvollziehbar speichern, welche Version von Code, Modell, Daten und Konfiguration zusammengehört.

Für produktive Systeme

"Das Modell läuft auf meinem Rechner" ist erst der Anfang. Ein echtes KI-System braucht Datenqualität, Tests, Sicherheit, Monitoring, Versionierung, Betrieb, Datenschutz und klare menschliche Verantwortlichkeiten.

LLMOps

LLMOps ist die auf große Sprachmodelle zugeschnittene Variante von MLOps. Zusätzlich spielen Prompts, RAG-Dokumente, Vektordatenbanken, Tokenkosten, Kontextlänge, Halluzinationen und Modellprovider eine Rolle.

Halluzination

Eine Halluzination ist eine inhaltlich falsche oder unbelegte Ausgabe, die sprachlich überzeugend klingt. Bei einem Klassifikationssystem äußert sich das eher als falsches Label oder als unerlaubte freie Begründung. Deshalb braucht man Evaluation und gegebenenfalls Ausgabevalidierung.

Explainability / Erklärbarkeit

Erklärbarkeit bedeutet, nachvollziehbar zu machen, warum ein System eine Entscheidung getroffen hat. Bei LLMs ist das schwieriger als bei einfachen Regeln. Wichtig: Eine vom LLM generierte Begründung ist nicht automatisch eine echte technische Erklärung seiner internen Entscheidung.

22. Lernfragen mit Antworten

Was ist ein Transformer?

Eine neuronale Netzwerkarchitektur für Sequenzen. Kernmechanismus ist Attention, mit der Beziehungen zwischen Tokens gewichtet werden.

Was ist ein Token?

Eine kleine Texteinheit, die der Tokenizer in eine Zahl übersetzt.

Was macht ein LLM?

Es berechnet auf Basis des bisherigen Kontexts Wahrscheinlichkeiten für weitere Tokens und erzeugt daraus Text.

Was ist Fine-Tuning?

Weitertraining eines vortrainierten Modells für eine konkretere Aufgabe oder Domäne.

Was ist PEFT?

Oberbegriff für Fine-Tuning-Methoden, bei denen nur relativ wenige Parameter angepasst werden.

Was ist LoRA?

Eine PEFT-Methode, die kleine trainierbare Matrizen an bestimmte Modellschichten hängt.

Was bedeutet r bei LoRA?

Rank bzw. Kapazität der kleinen LoRA-Matrizen.

Was ist ein Adapter?

Die gespeicherten LoRA-Zusatzgewichte.

Was ist Merge?

LoRA-Änderungen werden mit dem Basismodell zu einem vollständigen Modell zusammengeführt.

Was ist GGUF?

Ein Dateiformat für effiziente lokale Ausführung von Sprachmodellen, z. B. mit llama.cpp/Ollama.

Was ist Quantisierung?

Gewichte mit weniger Bits darstellen, um Speicher und Rechenaufwand zu reduzieren.

Was ist Inferenz?

Ein trainiertes Modell anwenden, ohne seine Gewichte zu ändern.

Was ist eine Epoche?

Ein kompletter Durchlauf durch alle Trainingsbeispiele.

Was ist die Learning Rate?

Schrittgröße der Gewichtsänderung beim Training.

Was ist Loss?

Trainingsfehler, den das Optimierungsverfahren möglichst reduzieren will.

Was ist Overfitting?

Das Modell ist auf Trainingsdaten gut, aber auf neuen Daten schlecht.

Was ist RAG?

Relevante externe Informationen suchen und dem LLM vor der Antwort als Kontext geben.

Was ist Embedding?

Ein Zahlenvektor, der semantische Eigenschaften eines Textes repräsentiert.

Was ist Ollama?

Eine lokale Laufzeit, über die Sprachmodelle geladen und per CLI/API ausgeführt werden können.

Warum testen wir auf unbekannten Daten?

Nur so sehen wir, ob das Modell generalisiert statt Trainingsbeispiele auswendig zu reproduzieren.

23. Kompakter Spickzettel

Begriff	Einfach erklärt
LLM	Großes Sprachmodell
Transformer	Architektur hinter modernen LLMs
Attention	Gewichtet Beziehungen zwischen Tokens
Token	Kleine Texteinheit
Tokenizer	Text <-> Token-IDs
Embedding	Bedeutung als Zahlenvektor
Prompt	Anweisung/Eingabe
Context Window	Maximal berücksichtigte Tokenmenge
Inference	Modell anwenden
Fine-Tuning	Modell weitertrainieren
PEFT	Ressourcensparendes Fine-Tuning

LoRA	Kleine Zusatzmatrizen trainieren
Adapter	Gespeicherte LoRA-Gewichte
Merge	Adapter + Basismodell zusammenführen
Epoch	Ein Durchlauf durch Trainingsdaten
Batch	Gruppe gleichzeitig verarbeiteter Beispiele
Learning Rate	Schrittgröße beim Lernen
Loss	Trainingsfehler
Overfitting	Auswendig gelernt, schlecht auf neuen Daten
Safetensors	Format für Modellgewichte
GGUF	Format für lokale effiziente Inferenz
Quantisierung	Weniger Bits pro Gewicht
Ollama	Lokale Modell-Laufzeit
RAG	Suchen + Kontext + LLM
Ground Truth	Fachlich korrektes Label
Precision	Wie sauber sind positive Treffer?
Recall	Wie viel Relevantes wurde gefunden?
F1	Balance zwischen Precision und Recall
MLOps	Betrieb und Lebenszyklus von ML-Systemen
Drift	Produktive Daten/Welt verändern sich

Die wichtigste Denkweise

Ein KI-System ist keine einzelne "magische KI". Es ist eine Kette aus Daten, Tokenizer, Modell, Training, Konfiguration, Dateiformaten, Laufzeit, API, Evaluation und Betrieb. Wenn du diese Kette einzeln erklären kannst, verstehst du bereits sehr viel von praktischer KI-Entwicklung.

Unser Use Case als ein Satz

Wir passen ein lokales Sprachmodell mit LoRA an eine binäre Textklassifikation an, führen Adapter und Basismodell zusammen, konvertieren das Ergebnis nach GGUF, stellen es mit Ollama lokal bereit und vergleichen die Qualität anhand unabhängiger Testdaten.